

1. Burn Xiaozhi Firmware

1. Burn Xiaozhi Firmware

- 1. Xiaozhi AI Overview
 - What is AI Xiaozhi?
 - Implemented Features Overview
 - Tutorial Coverage
- 2. Configuration and Preparation
 - 1. Prerequisites
 - 2. Clone Source Repository
 - 3. Install Third-Party Component Dependencies
- 4. Configuration and Compilation
 - Step 1: Set Target Chip
 - Step 2: menuconfig Configuration
 - Open menuconfig
 - ① Board Model
 - ② Flash Size
 - ③ Partition Table
 - ④ PSRAM
 - Save and Exit
 - Step 3: First Compilation
- 5. Compilation and Flashing
- 6. Comparison of Two Flashing Methods
 - Method 1: Self-Compile and Flash (Sections 2~5 above)
 - Method 2: Use Pre-compiled Firmware Direct Flash
- 7. Verification: First Boot Logs

1. Xiaozhi AI Overview

What is AI Xiaozhi?

AI Xiaozhi is an open-source AI voice chat robot based on the ESP32 series chip, utilizing the AI capabilities of large language models like Qwen/DeepSeek, and implementing voice interaction and device control through the **MCP (Model Context Protocol)**. It is essentially a "talking smart terminal": your words are collected, encoded by the device, and sent to the cloud AI; the voice returned from the cloud is decoded and played locally, enabling free human-machine dialogue.

This project is open-sourced by Brother Xia Ge and released under the MIT license. The current stable version is v2.2.6, requiring ESP-IDF $\geq$ 5.5.2.

Implemented Features Overview

Feature	Description
Wi-Fi / 4G Networking	Supports dual-mode networking via Wi-Fi and ML307 Cat.1 4G module
Offline Voice Wake-up	Based on ESP-SR, supports custom wake words (e.g., "Xiaozhi Xiaozhi")

Feature	Description
Dual Protocol Communication	WebSocket or MQTT+UDP
OPUS Audio Codec	Low-latency, high-compression voice transmission
ASR + LLM + TTS	Streaming speech recognition → LLM inference → speech synthesis
Voiceprint Recognition	Identifies current speaker based on 3D Speaker
OLED / LCD Display	Supports expressions, status, and dialogue text display
MCP Device Control	Control volume, lights, motors, GPIO, etc. via MCP protocol
Multi-language	Chinese, English, Japanese
Multi-platform	ESP32-C3, ESP32-S3, ESP32-P4
OTA Upgrade	Supports Wi-Fi wireless firmware upgrade

Tutorial Coverage

This tutorial breaks down the process of building Xiaozhi AI from scratch to free dialogue into five progressive chapters, each completing a verifiable milestone:

Chapter	Content	After Completion
1. Burn Xiaozhi Firmware	Get source code, configure ESP-IDF environment, install component dependencies	Successfully compile and flash official firmware
2. Network and Configuration	Wi-Fi provisioning, WebSocket/MQTT protocol selection, server connection	Device successfully connects to network
3. AI Agent Settings	Register AI platform, get API Key, configure LLM parameters	Device has an AI brain
4. Program Analysis	Project architecture, state machine, audio pipeline, MCP service, display system	Understand code for secondary development
5. Free Chat	Wake-up interaction, dialogue flow, custom configuration, troubleshooting	Freely chat with Xiaozhi and customize features

Learning Suggestion: If this is your first time working with ESP32 or AI hardware development, it is recommended to follow the chapters sequentially.

2. Configuration and Preparation

1. Prerequisites

Before starting, ensure you have completed the following preparations (see basic tutorial for details):

Preparation	Requirement	Reference Tutorial
ESP-IDF Environment	6.0.1 ≥ version ≥ 5.5.2	[1. Environment Setup]
Target Chip	ESP32-S3 (used in this tutorial)	—
Speaker + MIC	Onboard ES8311 audio codec	—
USB Data Cable	Type-C, supports data transfer	—

Why can't ESP-IDF 6.0.x be used? ESP-IDF 6.0.x introduced many breaking API changes (such as `esp_netif` interface refactoring, Wi-Fi driver API deprecation), while Xiaozhi firmware (v2.2.6) is written based on ESP-IDF 5.x, with many dependencies on old APIs and build systems (Kconfig/CMake macros). Compiling on 6.0.x will directly report errors, requiring source code modifications to adapt throughout—extremely workload-intensive and prone to introducing bugs. Therefore, official support is only for 5.x series ≥ 5.5.2.

Recommended: Use VSCode + ESP-IDF Extension

2. Clone Source Repository

Open terminal, enter your project directory, and execute clone:

```
# Clone AI Xiaozhi main repository
git clone https://github.com/78/xiaozhi-esp32.git -b v2.2.6
cd xiaozhi-esp32

# Check current version
git describe --tags
```

After cloning, the project root directory structure is as follows:

```
xiaozhi-esp32-main/
├─ CMakeLists.txt      ← Project entry (project name "xiaozhi", version 2.2.6)
├─ main/
│   ├─ main.cc         ← app_main() entry
│   ├─ application.cc/h ← Application main control logic
│   ├─ audio/          ← Audio codec, wake word, audio pipeline
│   │   ├─ codecs/      ← ES8311/ES8374/ES8388 and other codec drivers
│   │   ├─ processors/  ← Audio processors (AFC, AEC, VAD, etc.)
│   │   └─ wake_words/  ← Wake word models
│   ├─ boards/         ← 70+ board configurations
│   ├─ protocols/      ← WebSocket / MQTT protocol implementation
│   ├─ display/        ← OLED/LCD display drivers
│   ├─ led/            ← LED light effect control
│   ├─ assets/         ← Multi-language resources (fonts, expressions, voice prompts)
│   ├─ mcp_server.cc/h ← Device-side MCP service
│   ├─ ota.cc/h        ← OTA online upgrade
│   └─ settings.cc/h   ← NVS persistent configuration
├─ config/            ← Kconfig menu configuration
├─ partitions/        ← Partition table definition
└─ scripts/           ← Auxiliary scripts
```

3. Install Third-Party Component Dependencies

AI Xiaozhi depends on many third-party libraries in the ESP-IDF component registry. The dependency declaration file is located at `main/idf_component.yml`.

Core Dependency List:

Component	Version	Purpose
<code>espressif/esp_codec_dev</code>	~1.5.6	ES8311 and other audio codec drivers
<code>espressif/esp_audio_codec</code>	~2.4.1	Audio codec framework
<code>espressif/esp_audio_effects</code>	~1.2.1	Audio effects processing
<code>espressif/esp-sr</code>	~2.3.0	Speech recognition (wake word)
<code>78/esp-wifi-connect</code>	~3.1.3	Wi-Fi smart provisioning
<code>78/xiaozhi-fonts</code>	~1.6.0	Xiaozhi dedicated font package
<code>78/esp-m1307</code>	~3.6.5	ML307 4G module driver
<code>lvgl/lvgl</code>	~9.5.0	LVGL graphics library (display)
<code>esp_lvgl_port</code>	~2.7.2	LVGL ESP porting layer
<code>espressif/led_strip</code>	~3.0.2	WS2812 and other LED drivers
<code>espressif/button</code>	~4.1.5	Button driver
<code>espressif/esp32-camera</code>	^2.1.6	Camera support (some board models)

Tip: The component manager automatically downloads third-party dependencies from remote repositories to the `managed_components/` directory when executing `idf.py build`. First build requires internet; subsequent builds use cache.

```
# main/idf_component.yml (key excerpt)
dependencies:
  espressif/esp-sr: ~2.3.0
  78/esp-wifi-connect: ~3.1.3
  espressif/esp_audio_codec: ~2.4.1
  espressif/esp_codec_dev: ~1.5.6
  78/xiaozhi-fonts: ~1.6.0
# ... more dependencies in source file
idf:
  version: '>=5.5.2'
```

4. Configuration and Compilation

Step 1: Set Target Chip

```
idf.py set-target esp32s3
```

After execution, IDF automatically generates `sdkconfig` default configuration.

Step 2: menuconfig Configuration

`menuconfig` is a GUI configuration tool provided by ESP-IDF, allowing you to modify project settings, component options, and partition tables without directly editing code. The configuration items that need modification for this project and their menuconfig paths are as follows (automatically generated default items are not listed).

Operation tips: **↑↓** move cursor, **Space** check/switch options, **Enter** enter submenu/edit text, **Q** exit current layer (top layer Q prompts save, **Y** save and exit / **N** discard), **S** save but don't exit, **?** view help.

Open menuconfig

Execute in project root directory (refer to **3. Serial Communication and Storage System → 4. PSRAM External Memory Experiment**):

```
idf.py menuconfig
```

Wait for terminal to load configuration items (first open takes about 5-10 seconds to compile mconf tool).

① Board Model

```
Top ▸ Xiaozhi
    └─ Board Type ───────────────────→ (X) esp-box-lite
```

#	Operation	Description
1	Enter <code>Xiaozhi</code> → select <code>Board Type</code> , press Space to select corresponding board	e.g., <code>esp-box-lite</code>

② Flash Size

```
Top ▸ Serial flasher config
    └─ Flash size ───────────────────→ (X) 16 MB
```

#	Operation	Description
1	<code>Serial flasher config</code> → select <code>Flash size</code> , press Space to switch to <code>16 MB</code>	Match onboard 16MB SPI Flash

③ Partition Table

Top ▶ Partition Table

└ Partition Table

└ (X) Custom partition table CSV

└ (partitions.csv) Custom partition CSV file

→ partitions/v2/partition-table.csv

#	Operation	Description
1	Enter Partition Table → select Partition Table, press Space to switch to (X) Custom partition table CSV	Enable custom partition table
2	Enter Custom partition CSV file, press Enter to edit, change to partitions/v2/partition-table.csv	Point to Xiaozhi partition CSV file

④ PSRAM

Top ▶ Component config

└ ESP PSRAM

└ [*] Support for external, SPI-connected RAM

#	Operation	Description
1	Component config → ESP PSRAM → check [*] Support for external, SPI-connected RAM	Enable 8MB PSRAM; AI voice processing requires large memory

PSRAM is a **mandatory** option for AI Xiaozhi—audio codec buffer, LVGL display buffer, and wake word model loading all depend on PSRAM. Not enabling will cause compilation failure or runtime out-of-memory.

Save and Exit

Press **Q** → **Y** in sequence to save and exit; menuconfig automatically updates the `sdkconfig` file.

Configuration Quick Reference (can also be preset via `sdkconfig.defaults` to avoid repeated configuration):

Configuration Item	Value	Menu Location	Description
XIAOZHI_BOARD_TYPE	esp-box-lite	Xiaozhi	Board model
ESPTOOLPY_FLASHSIZE_16MB	y	Serial flasher config	Flash size 16MB
PARTITION_TABLE_CUSTOM	y	Partition Table	Enable custom partition table
PARTITION_TABLE_CUSTOM_FILENAME	"partitions/v2/partition-table.csv"	Partition Table	Partition CSV file path
SPIRAM	y	ESP PSRAM	Enable external PSRAM

```
# sdkconfig.defaults - AI Xiaozhi Required Configuration
# Flash size
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y

# Custom partition table
CONFIG_PARTITION_TABLE_CUSTOM=y
CONFIG_PARTITION_TABLE_CUSTOM_FILENAME="partitions/v2/partition-table.csv"

# PSRAM / External RAM
CONFIG_SPIRAM=y
```

The above `sdkconfig.defaults` only lists required items. Wi-Fi and server configuration can be done in Chapters 2 and 3 via `menuconfig` or dynamic provisioning.

Step 3: First Compilation

```
idf.py build
```

During first compilation, the ESP-IDF component manager automatically downloads all third-party dependencies declared in `idf_component.yml`. This process takes a few minutes; please be patient.

Compilation Success Indicator:

```
Project build complete.
```

If compilation reports errors, common issues:

Error Message	Cause	Solution
<code>fatal error: xxx.h: No such file</code>	Component not downloaded	Execute <code>idf.py clean</code> then <code>idf.py build</code> again
<code>CMake Error: Version mismatch</code>	ESP-IDF version too low	Upgrade to $\geq 5.5.2$
<code>No partition table found</code>	Partition table path error	Confirm partition table path in <code>menuconfig</code>
<code>Target mismatch</code>	Chip target error	Execute <code>idf.py set-target esp32s3</code>

5. Compilation and Flashing

ESP-IDF Flashing Process: See `ESP-IDF Development Tutorial` → 1. Before Development → 3. Build and Flash.

6. Comparison of Two Flashing Methods

There are two ways to get Xiaozhi firmware, choose based on your situation:

	Method 1: Self-Compile and Flash	Method 2: Use Pre-compiled Firmware Direct Flash
Prerequisites	ESP-IDF environment $\geq$ 5.5.2, Git	Only serial driver and flashing tool required
Steps	Clone source $\rightarrow$ menuconfig $\rightarrow$ <code>idf.py build</code> $\rightarrow$ <code>idf.py flash</code>	Download <code>.bin</code> file $\rightarrow$ flashing tool one-click flash
Time	First 30~60 minutes (including component download + compilation)	3~5 minutes
Target Users	Developers with ESP-IDF development environment, need custom configuration	Beginners, no development environment, want quick experience
Customization	Can modify wake word, board model, server, and all configurations	Uses preset default configuration (use Method 1 for customization)

Suggestion: Beginners or quick experience $\rightarrow$ use Method 2 direct flash. For modifying configuration or secondary development later $\rightarrow$ return to Method 1 self-compilation.

Method 1: Self-Compile and Flash (Sections 2~5 above)

Follow the complete flow of **2. Clone Source Repository** $\rightarrow$ **3. Install Dependencies** $\rightarrow$ **4. Configuration and Compilation** $\rightarrow$ **5. Compilation and Flashing**. The advantage is complete control, with every menuconfig option customizable.

Method 2: Use Pre-compiled Firmware Direct Flash

This tutorial provides pre-compiled firmware package; download, extract, and flash directly without setting up ESP-IDF environment. **ESP-IDF Flashing Process:** See `ESP-IDF Development Tutorial` $\rightarrow$ **1. Before Development** $\rightarrow$ **3. Build and Flash**. Flash addresses for each file are as follows:

Pre-compiled firmware uses default configuration: board `esp-box-lite`, server `xiaozhi.me`, protocol `websocket`. For custom configuration (wake word, board model, etc.), use Method 1 self-compilation.

7. Verification: First Boot Logs

After successful flashing, reset the development board; serial output should be similar to:

```
I (77) esp_psram: Found 8MB PSRAM device
I (80) esp_psram: Speed: 80MHz
I (102) app_init: Project name:      xiaozhi
I (106) app_init: App version:      2.2.6
I (119) app_init: ESP-IDF:          v5.5.4
I (162) esp_psram: Adding pool of 8192K of PSRAM memory to heap allocator
I (212) main_task: Calling app_main()
I (220) Board: UUID=80fb638b... SKU=esp-box-lite
I (324) OledDisplay: Initialize LVGL
I (327) StateMachine: State: unknown -> starting
I (346) Es8311AudioCodec: Duplex channels created
I (355) Es8311AudioCodec: Es8311AudioCodec initialized
```

```
I (355) AudioCodec: Audio codec started
I (357) MCP: Add tool: self.get_device_status
I (357) MCP: Add tool: self.audio_speaker.set_volume
I (362) MCP: Add tool: self.screen.set_theme
... (Total 8 MCP tools registered)
I (398) WifiManager: Initializing...
I (537) wifi:mode : sta (a4:cb:8f:c6:b8:34)
I (1477) wifiBoard: EnterWifiConfigMode called
I (1499) StateMachine: State: starting -> wifi_configuring
I (1537) wifiConfigurationAp: Access Point started with SSID Xiaozhi-B835
I (1553) esp_netif_lwip: DHCP server started on interface WIFI_AP_DEF with IP: 192.168.4.1
W (1554) Application: Alert [gear] Config mode: phone connects to hotspot xiaozhi-B835,
browser visits http://192.168.4.1
I (1579) AudioCodec: Set output enable to true
I (1580) I2S_IF: STD: TX, sample_rate_hz: 16000
I (1599) I2S_IF: STD: RX, sample_rate_hz: 16000
```

Key Log Nodes:

Log Line	Stage	Description
Found 8MB PSRAM device	PSRAM mount	Detected 8MB Octal PSRAM and added to heap allocator
App version: 2.2.6	Version confirmation	Project name xiaozhi, based on ESP-IDF v5.5.4
Board: SKU=esp-box-lite	Board identification	UUID + SKU confirms board model
OledDisplay: Initialize LVGL	Display system	LVGL task started, OLED screen ready
StateMachine: unknown -> starting	State machine	Device entering startup state
Es8311AudioCodec initialized	Audio codec	ES8311 duplex channel created, Slave mode
MCP: Add tool: ...	MCP tool registration	Registered 8 MCP tools (system status, volume, theme, etc.)
wifiBoard: EnterWifiConfigMode	Enter provisioning	No saved Wi-Fi credentials, automatically create AP hotspot
State: starting -> wifi_configuring	State transition	Device entering Wi-Fi config state
SSID xiaozhi-B835, IP: 192.168.4.1	Provisioning hotspot ready	Phone connects to this hotspot, browser visits 192.168.4.1 to complete Wi-Fi config

Log Line	Stage	Description
I2S_IF: TX/RX, sample_rate_hz: 16000	Audio channel ready	I2S full-duplex 16kHz ready, recording + playback channels opened

First boot automatically enters **provisioning mode** due to no Wi-Fi credentials; device creates hotspot named `xiaozhi-xxxx`. After phone connects, browser visits `192.168.4.1` to complete Wi-Fi configuration. See [2. Network and Configuration](#).